Lekcja 14. Funkcje członkowskie typu inline. Wskaźniki obiektów

Przedmiot: Programowanie aplikacji Klasa: 4 technikum Zawód: technik programista (351406) Kwalifikacja: INF.04 - Projektowanie, programowanie i testowanie aplikacji

Powiązanie z podstawą programową: INF.04.4 Programowanie obiektowe - uczeń korzysta z klas i obiektów, definiuje pola i metody, tworzy obiekty oraz posługuje się składowymi obiektów.

1. Cele lekcji

• wyjaśnić zasadę działania funkcji członkowskich typu inline;

• zdefiniować i wywołać krótką metodę inline w klasie;

• utworzyć wskaźnik do obiektu i posługiwać się operatorami & oraz ->;

• rozróżnić obiekt tworzony automatycznie na stosie od obiektu tworzonego dynamicznie na stercie;

• utworzyć obiekt za pomocą new i poprawnie zwolnić pamięć poleceniem delete.

2. Funkcje członkowskie typu inline

Funkcja członkowska typu inline jest metodą klasy oznaczoną słowem kluczowym inline. W materiale źródłowym opisano ją jako funkcję, której kod może zostać wstawiony w miejscu wywołania podczas kompilacji. Rozwiązanie to ma sens przede wszystkim dla krótkich, prostych metod wykonywanych często.

Ważne: funkcja inline nadal jest zwykłą metodą klasy. Może odwoływać się do pól obiektu i jest wywoływana przez obiekt tak samo jak pozostałe metody.

Podręcznik zaleca, aby funkcje inline były krótkie i nie zawierały złożonych pętli, rozbudowanych instrukcji warunkowych, instrukcji wyboru ani skoków. Definicja funkcji musi być dostępna przed jej użyciem.

Uzupełnienie współczesne: słowo inline nie wymusza optymalizacji polegającej na wstawieniu kodu funkcji. Ostateczną decyzję podejmuje kompilator. Podręcznik podaje również ograniczenie dotyczące zmiennych statycznych; w nowoczesnym C++ nie jest to bezwzględny zakaz języka, dlatego traktujemy tę uwagę jako zalecenie dydaktyczne z materiału źródłowego.

2.1. Przykład 11.8 - klasa Pokoj

```
#include <iostream>
#include <cmath>
using namespace std;

class Pokoj {
public:
    float dlugosc;
    float szerokosc;

    inline int dlugoscCalc() {
        return round(dlugosc);
    }

    inline int szerokoscCalc() {
        return round(szerokosc);
    }
};

int main() {
    Pokoj pokoj;
    pokoj.dlugosc = 1.51;
    pokoj.szerokosc = 2.51;

    cout << "Dlugosc: " << pokoj.dlugoscCalc() << endl;
    cout << "Szerokosc: " << pokoj.szerokoscCalc() << endl;
    return 0;
}
```

W klasie Pokoj zdefiniowano dwie metody inline. Obie korzystają bezpośrednio z pól obiektu i zwracają ich wartości zaokrąglone do najbliższej liczby całkowitej.

Ćwiczenie z podręcznika polega na rozbudowaniu przykładu o funkcje zaokrąglające długość i szerokość odpowiednio w górę i w dół, np. z użyciem funkcji ceil() i floor().

3. Wskaźniki do obiektów

Wskaźnik do obiektu przechowuje adres obiektu określonej klasy. Zasada działania jest analogiczna do wskaźników do struktur, ale dostęp do pól i metod obiektu odbywa się najczęściej za pomocą operatora strzałkowego ->.

Zapis	Znaczenie
`Pracownik *p;`	deklaracja wskaźnika do obiektu klasy Pracownik
`p = nullptr;`	wskaźnik nie wskazuje żadnego obiektu
`p = &pracownik;`	przypisanie adresu istniejącego obiektu
`p->imie`	dostęp do pola obiektu przez wskaźnik
`p->wyswietlDane()`	wywołanie metody obiektu przez wskaźnik

Operator & pobiera adres obiektu. nullptr oznacza, że wskaźnik nie wskazuje aktualnie żadnego obiektu. Operator -> służy do odwołania do składowej obiektu przez wskaźnik.

3.1. Przykład 11.9 - obiekt na stosie i wskaźnik

```
class Pracownik {
public:
    string imie;
    string nazwisko;

    void wyswietlDane() {
        cout << "Imie: " << imie << endl;
        cout << "Nazwisko: " << nazwisko << endl;
    }
};

int main() {
    Pracownik pracownik;

    Pracownik *w_pracownik = nullptr;
    w_pracownik = &pracownik;

    w_pracownik->imie = "Jan";
    w_pracownik->nazwisko = "Kowalski";
    w_pracownik->wyswietlDane();
    return 0;
}
```

Obiekt pracownik został utworzony automatycznie. Wskaźnik w_pracownik otrzymuje jego adres. Do pól i metod obiektu odwołujemy się przez wskaźnik i operator ->. Nie używamy tutaj delete, ponieważ obiekt nie został utworzony przez new.

Równoważny zapis: w_pracownik->imie oznacza (*w_pracownik).imie. Operator -> jest po prostu wygodniejszą formą dostępu.

3.2. Przykład 11.10 - obiekt dynamiczny na stercie

```
int main() {
    Pracownik *w_pracownik = nullptr;

    w_pracownik = new Pracownik();
    w_pracownik->imie = "Jan";
    w_pracownik->nazwisko = "Kowalski";
    w_pracownik->wyswietlDane();

    delete w_pracownik;
    w_pracownik = nullptr;
```

```
    return 0;
}
```

Instrukcja new tworzy obiekt dynamicznie i zwraca jego adres, który zapisujemy we wskaźniku. Obiekt taki istnieje do chwili wykonania delete lub zakończenia procesu. Po zwolnieniu pamięci warto przypisać do wskaźnika nullptr.

Wyciek pamięci powstaje wtedy, gdy pamięć przydzielona przez new nie zostanie zwolniona. Wiszący wskaźnik to wskaźnik przechowujący adres pamięci, która została już zwolniona.

4. Stos a sterta - porównanie

Cecha	Obiekt na stosie	Obiekt dynamiczny na stercie
Tworzenie	Pracownik p;	Pracownik *p = new Pracownik();
Dostęp	p.imie	p->imie
Czas życia	wynika z zakresu	do delete lub zakończenia programu
Zwalnianie pamięci	automatyczne	ręczne: delete p;

Uzupełnienie współczesne: w nowoczesnym C++ bezpośrednie używanie new i delete ogranicza się do sytuacji, w których jest to rzeczywiście potrzebne. W kodzie produkcyjnym często wykorzystuje się automatyczny czas życia obiektów oraz inteligentne wskaźniki. Na tej lekcji surowe wskaźniki są potrzebne do zrozumienia mechanizmu adresowania i pamięci.

5. Najczęstsze błędy

- użycie wskaźnika bez ustawienia go na poprawny adres;
- wywołanie składowej przez . zamiast -> (lub odwrotnie);
- wykonanie delete dla obiektu, który nie został utworzony przez new;
- brak delete po new - wyciek pamięci;
- dostęp przez wskaźnik po zwolnieniu obiektu;
- zbyt rozbudowane funkcje oznaczane jako inline bez uzasadnienia.

6. Podsumowanie

Funkcje członkowskie typu inline są krótkimi metodami, które mogą zostać zoptymalizowane przez kompilator. Wskaźniki do obiektów umożliwiają operowanie na obiektach poprzez ich adresy. Dla istniejącego obiektu używamy operatora & do pobrania adresu, a przez wskaźnik odwołujemy się do składowych operatorem ->. Obiekty tworzone za pomocą new należy zwolnić przez delete.

Skróty i pojęcia: INF.04 - Projektowanie, programowanie i testowanie aplikacji; OOP (Object-Oriented Programming) - programowanie obiektowe; RAII (Resource Acquisition Is Initialization) - technika wiążąca czas życia zasobu z czasem życia obiektu w C++.